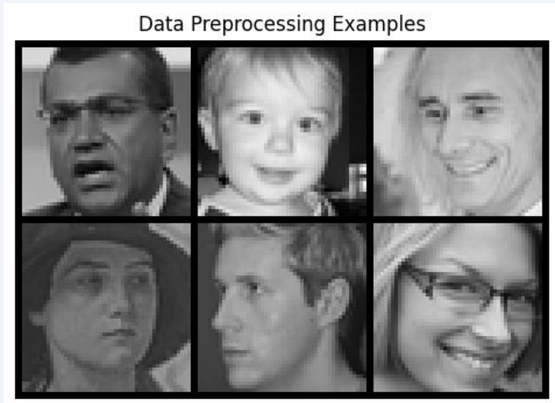
Facial Expression Classification Using CNN Model

Brandon Lequang

12/14/23

ABSTRACT

The following research poster explores the implementation and use of Convolutional Neural Networks (CNNs) for automatic human facial expression classification, taking advantage of the PyTorch library. The following project dives into the design process our extensive data preprocessing practices, CNN mode, and training model to optimize the classification performance to a reasonably high accuracy percentage. With this accuracy, the following model can effectively classify the following facial expressions: Happy, Angry, and Sad.

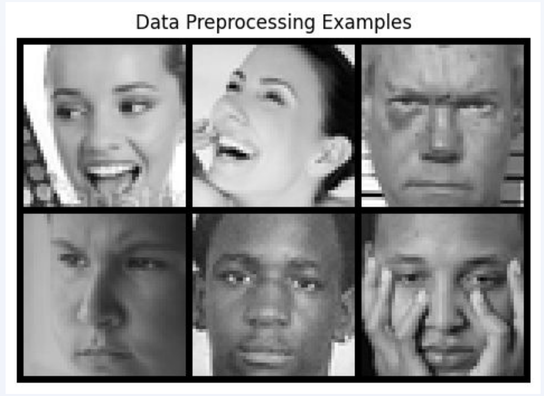


INTRODUCTION

In the following project, the methodology and results of implementing a convolutional neural network and using it to train a model targeted towards image facial expression classification. This project aims to practice the implementation of CNNs, alongside data preprocessing, adaptive learning rate, and dropout features.

METHODS

The following model was trained using a provided dataset that contained over 3500 grayscale images, which feature various facial expressions, ranging from Happy, sad, angry, etc.



The provided image dataset undergoes extensive Data Preprocessing, in which the the images are of optimal quality in order to be inputted for training through our convolutional neural network. Customizing the given datasets ensure that our neural network model and training model can handle the imputed images efficiently.

Within the convolution neural network, incorporating the use of multiple convolutional layers, batch normalization, and dropout methods reduces overfitting and improves learning.

The training loop utilizes the AdamW optimizer alongside a OneCycleLR scheduler in order to make the learning rate much more aggressive. Fifteen epochs are ran, and the final accuracy score will be calculated from the total average.

RESULTS

The CNN model undergone training using a dataset that consists of various facial expression images. The implemented training loop was performed over a defined number of 15 epochs:

```
Epoch 1, Validation Accuracy: 56.2596599690881%
Epoch 2, Validation Accuracy: 66.58423493044822%
Epoch 3, Validation Accuracy: 69.21174652241113%
Epoch 4, Validation Accuracy: 70.13910355486863%
Epoch 5, Validation Accuracy: 72.05564142194746%
Epoch 6, Validation Accuracy: 71.4064914992272%
Epoch 7, Validation Accuracy: 74.4049459041731%
Epoch 8, Validation Accuracy: 74.3740340030912%
Epoch 9, Validation Accuracy: 76.04327666151468%
Epoch 10, Validation Accuracy: 75.64142194744977%
Epoch 11, Validation Accuracy: 75.14683153013911%
Epoch 12, Validation Accuracy: 76.47604327666151%
Epoch 13, Validation Accuracy: 77.03245749613602%
Epoch 14, Validation Accuracy: 76.10510046367851%
Epoch 15, Validation Accuracy: 76.35239567233384%
```

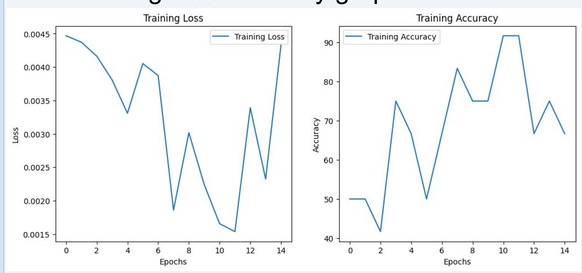
The initial validation accuracy was:

0.7635239567233385

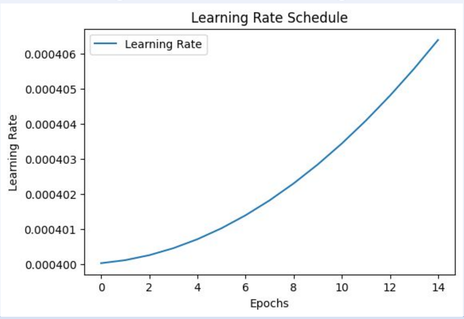
After performing test data predictions with the validation accuracy, the final validation accuracy is:

0.772

The training loss accuracy graphs:



The learning rate scheduler graph:



CONCLUSION

The outcomes of this project represent the learning ability that convolutional neural network models are capable of performing in relation to image classification training. The results show the model is capable of learning to an acceptable degree of accuracy. However, the results also show that there is more that can be done to optimize the model in terms of consistency and the generation of higher training accuracies.

REFERENCES

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436-444. https://doi.org/10.1038/nature14539

Abhinand A. (Year). In-Depth Guide to Convolutional Neural Networks. Kaggle. Available at: https://www.kaggle.com/code/abhinand05/in-depth-guide-to-convolutional-neural-networks

PyTorch. (2023). Datasets & Dataloaders. PyTorch Documentation. Retrieved from https://pytorch.org/docs/stable/data.html